

Operating Systems — Homework 2

Ex. 2.1 (15 pts) — Schedule diagrams: Round-Robin, FIFO and SJF

Task set (a_i, e_i) : $T_1(1, 10)$, $T_2(2, 4)$, $T_3(3, 1)$, $T_4(4, 6)$.

Conventions.

- Time is discrete: the task dispatched at instant t holds the CPU over $[t, t + 1)$, and a task with arrival time a_i is runnable at instant a_i .
- Nothing has arrived before $t = 1$, so $[0, 1)$ is idle under every policy; it is drawn, not cropped.
- FIFO is non-preemptive and dispatches in arrival order.
- SJF is the classical non-preemptive form: at each dispatch point it picks the arrived, unfinished task with the smallest *total* execution time, and that task then runs to completion.
- Round-Robin uses $q = 1$. Where a task arrives at the same instant that the running task exhausts its quantum, the arriving task is enqueued first and the preempted task then joins the tail of the ready queue.
- Every tie — equal bursts in SJF, equal standing in the ready queue — goes to the smaller index.

Total busy time is $10 + 4 + 1 + 6 = 21$ units, so with the single idle unit all three policies complete at $t = 22$.

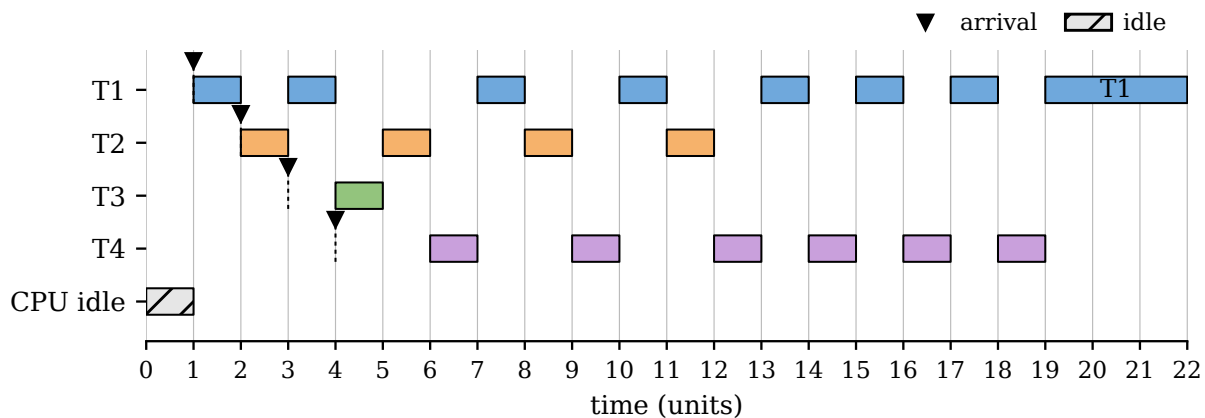


Figure 1: Round-Robin, $q = 1$.

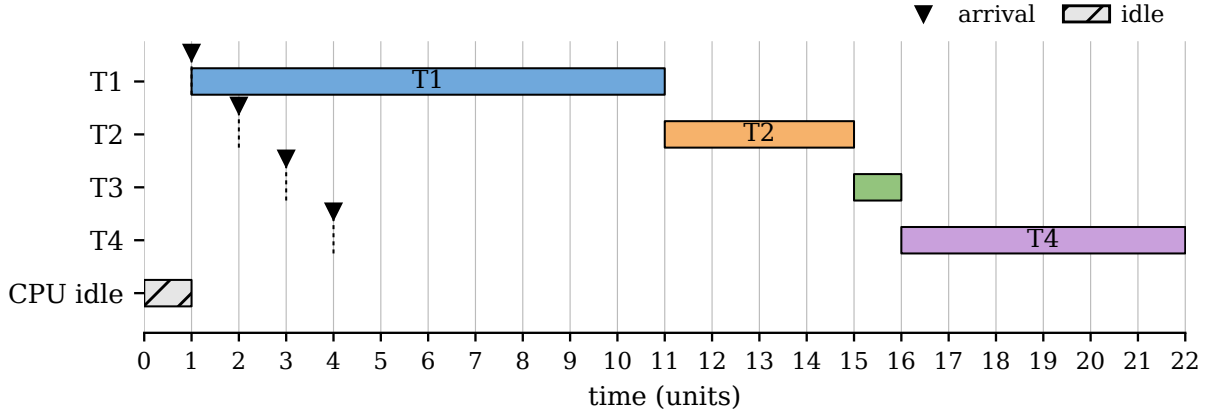


Figure 2: FIFO (non-preemptive).

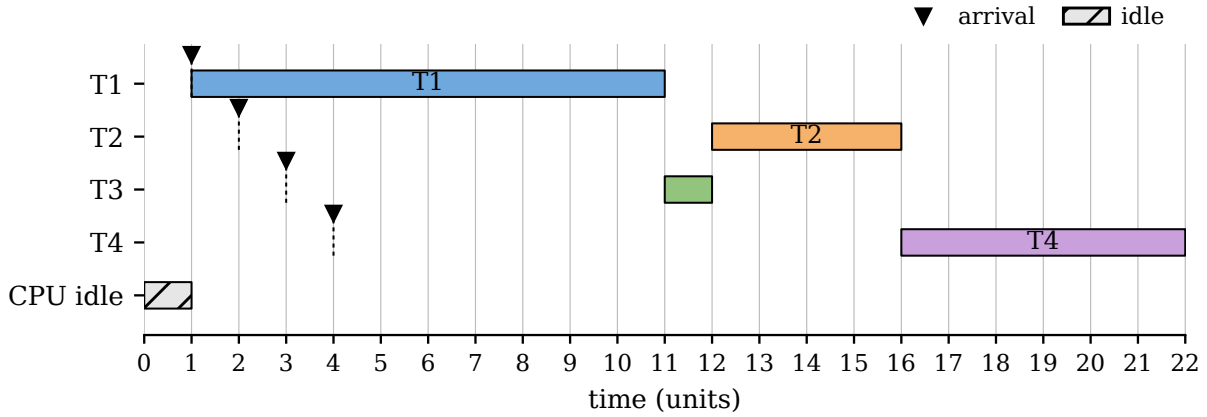


Figure 3: SJF (non-preemptive).

Task	a_i	e_i	Round-Robin ($q = 1$)			FIFO			SJF		
			C	T	W	C	T	W	C	T	W
T_1	1	10	22	21	11	11	10	0	11	10	0
T_2	2	4	12	10	6	15	13	9	16	14	10
T_3	3	1	5	2	1	16	13	12	12	9	8
T_4	4	6	19	15	9	22	18	12	22	18	12
average				12.00	6.75		13.50	8.25		12.75	7.50

Table 1: Completion C , turnaround $T = C - a_i$ and waiting $W = T - e_i$ times.

Round-Robin gives the lowest average turnaround and waiting times because the three short tasks interleave with T_1 instead of queueing behind its 10-unit burst, and FIFO the highest because T_3 waits 12 units to run for 1. The preemptive variant of SJF (SRTF) would give a different schedule:

T_2 preempts T_1 at $t = 2$ and T_3 preempts T_2 at $t = 3$.

Ex. 2.2 (5 pts) — Why process creation by a running process is the general case

The first two cases describe what *prompts* a process to be created; they do not describe a different creator. They are not alternatives to the third case, they are the third case seen from the user's side.

When someone double-clicks an icon or types a command, that person never touches the kernel. The click and the keystrokes are input events, and the kernel delivers them to a process that is *already running*: the desktop shell or window manager in the first instance, the command shell in the second. That process interprets the event, decides which program was meant, and calls `fork()` itself. The shell is the creator; the human only supplied the reason. Case 2 differs from case 1 only in where the trigger originated (another process, a timer, an arriving network connection) while the mechanism, a running process calling `fork()`, is unchanged.

All three cases therefore share a single creation primitive and differ only in what caused the running process to invoke it. Unix makes this literal: exactly one process is not born this way, the initial process (`init` or `systemd`, pid 1) that the kernel hand-builds at boot. Every other process is the child of some running process.

Ex. 2.3 (5 pts) — A program using `fork()` and `exec()` to run `ls`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     pid_t pid = fork();
9
10    if (pid < 0) {                /* case 1: fork failed */
11        perror("fork");
12        return EXIT_FAILURE;
13    }
14
15    if (pid == 0) {              /* case 2: we are the child */
16        char *argv[] = { "ls", "-l", NULL };
17        execvp("ls", argv);
18        /* only reached if execvp failed: the image was never replaced */
19        perror("execvp");
20        _exit(127);
21    }
22
23    /* case 3: we are the parent, pid holds the child's pid */
24    int status;
25    if (waitpid(pid, &status, 0) < 0) {
26        perror("waitpid");
27        return EXIT_FAILURE;
28    }
```

```

28     }
29
30     if (WIFEXITED(status))
31         printf("parent: child %d exited with status %d\n",
32                (int) pid, WEXITSTATUS(status));
33     else if (WIFSIGNALED(status))
34         printf("parent: child %d killed by signal %d\n",
35                (int) pid, WTERMSIG(status));
36
37     return EXIT_SUCCESS;
38 }

```

`fork()` returns twice, so its return value is the only thing that tells the two processes apart, and all three of its outcomes are handled. A negative value means no child was created. In the child (return value 0) `execvp` replaces the process image with `/usr/bin/ls` while keeping the same pid, searching `PATH` for the binary and passing a NULL-terminated `argv`. A successful `exec` never returns, so the lines after it run only on failure, where they report the error and leave via `_exit` rather than `exit` so the child does not flush the parent's copy of the `stdio` buffers. In the parent (return value the child's pid) `waitpid` blocks until that specific child terminates, which both sequences the output and reaps the child so it does not linger as a zombie.

Compiled with `gcc -Wall -Wextra -o forkexec forkexec.c` (gcc 12.2.0, Debian Linux) and run in a directory holding two small files. The actual output was:

```

total 8
-rw-r--r-- 1 xing xing 1 Sep 21 16:37 alpha.txt
-rw-r--r-- 1 xing xing 2 Sep 21 16:37 beta.txt
parent: child 100 exited with status 0

```

The first three lines come from `ls` after it has taken over the child, the last from the parent after `waitpid` returns. Setting `PATH` to a directory that does not exist exercises the failure branch: `execvp: No such file or directory, then parent: child 107 exited with status 127.`

Ex. 2.4 (5 pts) — Why a thread context switch is much cheaper than a process one

The claim holds for two threads *of the same process*. Threads belonging to different processes cost exactly what a process switch costs, because the expense comes from changing the address space, not from changing the scheduling entity.

Threads of one process share an address space, so the switch leaves the memory map untouched: the kernel never reloads the page-table base register (`CR3` on x86, `TTBR` on ARM) and so never invalidates the TLB. A process switch does both, and the register saving is the smaller half of its bill. The larger half is indirect and is paid after the switch returns: the incoming process resumes with a cold TLB and with caches still holding the outgoing process's lines, and stalls while both refill. Tagged TLBs (ARM ASIDs, x86 PCIDs) let entries for several address spaces coexist and soften this, but do not remove it, since a different working set still displaces cached data.

A thread switch still saves and restores the small per-thread state: the register file, including the program counter and stack pointer, and the thread's kernel stack and control block. Per-process

state such as the open-file table and signal dispositions is shared, so none of it is swapped. A switch between user-level threads avoids even the trap into the kernel.

Ex. 2.5 (20 pts) — Pseudo-code for `fork`, `exec`, `waitpid`, `exit`

Following the hint, each algorithm below is the *kernel-side implementation* of the call: the code the operating system runs when a user process traps into it. All four operate on the process table, whose entries are process control blocks (PCBs); `CURRENT` denotes the PCB of the calling process, and a negative return value is the usual `errno` convention.

Algorithm 1 `fork()` — create a copy of the calling process

```

1: procedure FORK()
2:   if no free process-table slot or no free pid then
3:     return -EAGAIN
4:   end if
5:    $c \leftarrow \text{ALLOCATEPCB}()$ ;  $c.\text{pid} \leftarrow \text{ALLOCATEPID}()$ 
6:   copy from CURRENT into  $c$ : uid, gid, cwd, root, umask,
     signal dispositions, resource limits, priority
7:    $c.\text{parent} \leftarrow \text{CURRENT}$ ; append  $c$  to CURRENT.children
8:    $c.\text{addrspace} \leftarrow \text{COPYONWRITESHARE}(\text{CURRENT.addrspace})$ 
9:   mark every writable page read-only in both processes ▷ copy on first write
10:  for all descriptors  $fd$  in CURRENT.fdtable do
11:     $c.\text{fdtable}[fd] \leftarrow \text{CURRENT.fdtable}[fd]$ 
12:    increment the reference count of that open-file entry
13:  end for
14:   $c.\text{cputime} \leftarrow 0$ ;  $c.\text{pending signals} \leftarrow \emptyset$ 
15:   $c.\text{savedregs} \leftarrow \text{CURRENT.savedregs}$  ▷ child resumes at the instruction after the trap
16:   $c.\text{savedregs.retval} \leftarrow 0$  ▷ what the child will see fork return
17:   $c.\text{state} \leftarrow \text{READY}$ ; ENQUEUE(ready queue, c)
18:  return  $c.\text{pid}$  ▷ what the parent will see fork return
19: end procedure

```

The two return-value assignments — 0 into the child’s saved register, the child’s pid into the parent’s — are the whole “`fork` returns twice” phenomenon: one call site, two saved register sets, two different values in the return register.

Algorithm 2 `exec(path, argv, envp)` — replace the caller’s program image

```

1: procedure EXEC(path, argv, envp)
2:   inode ← NAMEI(path); if inode = NULL then return -ENOENT
3:   if inode is not executable by CURRENT.uid then return -EACCES
4:   hdr ← READHEADER(inode)
5:   loader ← LOADERFOR(hdr.magic)           ▷ magic number selects a.out / ELF / script
6:   if loader = NULL then return -ENOEXEC
7:   kargv, kenvp ← COPYINTOKERNEL(argv, envp)           ▷ they live in the old address space
   — point of no return: every error above is still reportable to the caller —
8:   DESTROYADDRESSSPACE(CURRENT.addrspace)
9:   CURRENT.addrspace ← loader.MAPIMAGE(inode)           ▷ text r-x, data rw, bss zeroed
10:  create a fresh heap and user stack
11:  for all signals s caught by a user handler do
12:    disposition of s ← DEFAULT                       ▷ the handler’s code is gone
13:  end for
   ignored dispositions and the blocked signal mask are left untouched
14:  close every descriptor marked close-on-exec
15:  if inode is set-uid then CURRENT.euclid ← inode.owner
16:  sp ← PUSHONTONEWSTACK(kargv, kenvp)
17:  CURRENT.savedregs ← { pc : hdr.entry, sp : sp, others : 0 }
18:  return to user mode                               ▷ on success there is no caller left to return to
19: end procedure

```

Algorithm 3 `waitpid(pid, statusp, options)` — collect a terminated child

```

1: procedure WAITPID(pid, statusp, options)
2:   loop
3:     found ← FALSE
4:     for all c ∈ CURRENT.children matching pid do           ▷ pid = -1 matches any child
5:       found ← TRUE
6:       if c.state = ZOMBIE then
7:         if statusp ≠ NULL then COPYOUT(statusp, c.exitstatus)
8:         CURRENT.childeputime += c.cputime + c.childeputime
9:         remove c from CURRENT.children
10:        p ← c.pid; FREEPCB(c); FREEPID(p)                     ▷ the reaping step
11:        return p
12:      end if
13:    end for
14:    if found = FALSE then return -ECHILD
15:    if options contains WNOHANG then return 0
16:    if a signal is pending for CURRENT then return -EINTR
17:    SLEEP(CURRENT, its wait channel)                       ▷ a child’s exit wakes us; then rescan
18:  end loop
19: end procedure

```

Algorithm 4 `exit(status)` — terminate the calling process

```
1: procedure EXIT(status)
2:   for all open descriptors fd of CURRENT do
3:     CLOSE(fd)                                ▷ drops the reference count; the last close releases the file
4:   end for
5:   RELEASEADDRESSSPACE(CURRENT.addrspace)
6:   release cwd, root, timers and file locks
7:   for all c ∈ CURRENT.children do
8:     c.parent ← init; move c to init.children
9:     if c.state = ZOMBIE then SIGNAL(init, SIGCHLD); WAKEUP(init)    ▷ so init reaps it
10:  end for
11:  CURRENT.exitstatus ← status; record CPU time and accounting in the PCB
12:  CURRENT.state ← ZOMBIE                                ▷ PCB and pid stay allocated until reaped
13:  SIGNAL(CURRENT.parent, SIGCHLD); WAKEUP(CURRENT.parent)
14:  SCHEDULE()                                             ▷ picks another process; exit never returns
15: end procedure
```

How the four calls are inter-related

They are not four independent services but two pairs that bracket the life of a process.

`fork` is the only way a Unix system makes a new process, and `exec` is the only way a process changes the program it runs. Neither alone can start a new program: `fork` produces a second copy of the caller, `exec` replaces the caller's own image. Running a new program is therefore necessarily both of them, `fork` then `exec` in the child. The two algorithms are built to meet: `fork` gives the child a duplicate descriptor table and the parent's credentials, and `exec` keeps exactly that table, less the close-on-exec entries, while discarding everything else.

The split is what makes the arrangement useful. Between `fork` returning 0 and `exec` overwriting the image there is a window in which the child is still running the parent's code and can rearrange precisely the state that `exec` preserves. That window is where a shell implements `ls > out`: the child closes descriptor 1, reopens it on the file, then execs `ls`, which need know nothing about redirection.

`exit` and `waitpid` are the matching pair at the other end, and they are a pair for one reason: the exit status has to outlive the process that produced it. `exit` can free the address space and the descriptors, but it cannot free the PCB, because nobody has read the status out of it yet; so it stores the status there, marks the process ZOMBIE and signals the parent. `waitpid` is the other half of that transaction: it finds the zombie child, copies the status out, charges its CPU time to the parent, and only then releases the PCB and the pid. Until then the pid stays reserved, so “wait for process *p*” stays unambiguous.

A shell's main loop is the whole mechanism in one cycle: `fork`; the child `execs` the command; the parent blocks in `waitpid`; the child `exits` and becomes a zombie; the parent's `waitpid` returns with the status, frees the PCB, and prints the next prompt.

Ex. 2.6 (5 pts) — Zombie processes

A zombie is a process that has already run `exit()` but has not yet been reaped. Its address space, open files and kernel stack are gone; all that is left is its process-table entry, holding the exit status, the pid and the CPU time it used.

The kernel keeps that entry because the status belongs to the parent, and the child cannot report it once it no longer exists. The pid must stay allocated for the same reason: reusing it while the parent can still name it in `waitpid` would let the parent wait on an unrelated process.

The cost is one process-table slot and one pid — no memory, no CPU time, no descriptors. A zombie also cannot be killed, since it is already dead: `SIGKILL` is discarded, as no execution context is left to receive it. In `ps` it appears in state `Z`, with its command shown as `<defunct>`.

It disappears when the parent calls `wait` or `waitpid`, or when the parent itself exits and `init` inherits and reaps it. A long-lived parent that forks children and never waits leaks table entries until `fork` fails with `EAGAIN`, which is why such a parent should handle or explicitly ignore `SIGCHLD`.

An orphan is the complement: a child still *alive* when its parent dies. `init` adopts it and reaps it when it eventually exits.

Ex. 2.7 (22.5 pts) — Three papers from the “Scheduling” group

Paper 2-3 — Waldspurger and Weihl, *Lottery Scheduling* (OSDI 1994)

Summary. Lottery scheduling represents resource rights as *tickets* and settles each quantum by drawing one at random: a client holding t of T tickets wins with probability $p = t/T$, so throughput tracks the allocation, no non-zero holding can starve, and a reallocation takes effect at the very next decision. The abstractions built on the draw matter more than the draw itself. *Ticket transfers* hand a blocked client’s rights to whoever is computing on its behalf, dissolving priority inversion much as priority inheritance does. *Currencies* denominate tickets locally and convert through an exchange rate into a conserved base currency, so inflation inside one trust boundary cannot leak out. *Compensation tickets* multiply a client’s funding by $1/f$ when it used only a fraction f of its quantum, so a thread that yields early for I/O is not quietly punished. A Mach 3.0 prototype (25 MHz MIPS DECStation, 100 ms quantum) reports a 2:1 Dhrystone allocation yielding 25378 against 12619 iterations/s (2.01:1), an 8:3:1 database allocation whose response times while the best-funded client is active (17.19, 43.19 and 132.20 s) give relative speeds of 7.69:2.51:1, and currencies insulating one user’s inflation from another (aggregate ratio 1.01:1 before, 1.00:1 after).

Critical judgment. The price of randomisation is variance, and the paper meets that objection rather than ducking it: the one bad run, a 10:1 allocation observed at 13.42:1, is named as the exception to ratios otherwise “close to their corresponding allocations”, and Figure 5 goes on to plot 8-second windows inside a 200-second run. The weak step is the last one, where fairness at the timescale the introduction cares about — interactive control “at a time scale of milliseconds to seconds” — is obtained by argument instead of measurement: “if a scheduling quantum of 10 milliseconds were used instead of the 100 millisecond Mach quantum, the same degree of fairness would be observed over a series of subsecond time windows”. Nothing was ever run at 10 ms. The warrant offered for a fine quantum being cheap — “a tree-based lottery need only generate a random number and perform $\lg n$ additions and comparisons” — describes an implementation that was never

built, whereas every measurement in the paper comes from the unoptimised list-based prototype.

The overhead claim is thinner than its conclusion. The prototype ran 2.7% slower on three Dhrystone tasks and 0.8% slower on eight, and the authors honestly note the run-to-run standard deviation was as large as the effect; the one significant measurement, 1.7% *faster* on the database server, is attributed in a footnote to incidental cache and TLB locality. “Efficient lottery scheduling does not pose any challenging problems” therefore rests on data that mostly cannot separate the two schedulers. Currencies, finally, are what make inflation safe, and §5.5 does demonstrate the barrier containing one user’s inflation; what is missing is the control over *who* may inflate a currency, since the access-control lists that would supply it are named only as something “a complete lottery scheduling system should” provide.

Paper 2-4 — Waldspurger and Weihl, *Stride Scheduling* (MIT/LCS/TM-528, 1995)

Summary. Stride scheduling is the same authors’ deterministic replacement for their own dice. Each client holds a *stride* inversely proportional to its tickets and a *pass* value; the scheduler always runs the minimum-pass client and advances that client’s pass by its stride, in $O(\lg n_c)$ time with a priority queue. A partial quantum advances the pass by only $f \cdot \text{stride}$, while joins, leaves and ticket changes are handled by saving and rescaling the client’s unconsumed remainder against a smoothly advancing global pass. Because each client’s state is independent, the transfers, inflation and currencies of paper 2-3 all survive on the deterministic substrate. The error bounds are the argument: lottery scheduling’s absolute error grows as $O(\sqrt{n_a})$ in the number of allocations, whereas stride’s pairwise relative error never exceeds one quantum whatever n_a is. Absolute error can still reach $O(n_c)$ under skew, 101 clients at 100:1:…:1 giving an error of 50, which motivates *hierarchical* stride scheduling: a balanced tree of aggregated groups bounding absolute error to $O(\lg n_c)$, which measured 4.5 on that example. In simulation at 19:1 over one million allocations, the one-ticket client saw response times of $\mu = 20.00$, $\sigma = 0.01$ quanta under stride against $\mu = 20.13$, $\sigma = 19.64$ and a range of 1–194 quanta under lottery. Linux 1.1.50 prototypes held CPU ratios within 1% of ideal (3:1 gave 3.001:1) and UDP ratios within 5%, in under 300 lines each.

Critical judgment. The head-to-head case against lottery scheduling is made entirely in simulation. The prototypes are measured against the *ideal* ratio and against stock Linux, never against a lottery-scheduled kernel, so the headline claim of significantly improved accuracy over lottery is nowhere demonstrated on hardware. They also leave the modularity argument unexercised: Section 3 shows how transfers and currencies would be built, but the CPU prototype implements neither, so the abstractions that justify the design are never run. The overhead figure is softer than it reads, because the authors concede their prototype does a linear scan rather than the $O(\lg n_c)$ structure the complexity analysis assumes; the cheap measurement and the good bound describe different programs.

Against that, the paper is unusually candid and criticism should not pretend otherwise. It reports that the network result required lowering a hard-coded delay constant in `ttcp`, without which ratios were “consistently lower than specified”; it volunteers that its core algorithm turned out to be nearly VirtualClock and WFQ rediscovered; and its conclusion concedes that lottery scheduling is “conceptually simpler” and “effectively stateless” while stride demands careful state fixups on every dynamic change. History settled the trade: Linux’s CFS and its EEVDF successor both schedule by picking the minimum-virtual-time task and advancing it by a weight-scaled increment, stride’s rule

under another name, while no production kernel ever shipped the lottery.

Paper 2-1 — Buttazzo, *Rate Monotonic vs. EDF: Judgment Day* (Real-Time Systems 29(1), 2005)

Summary. This is a rebuttal rather than a new result. Buttazzo takes the five claims habitually used to prefer Rate Monotonic to Earliest Deadline First (simpler implementation, lower overhead, easier analysis, more predictable overload, less jitter) and answers each with theory, counterexamples and simulation. He grants RM one real point: on a commercial kernel with at most 256 priority levels, a new deadline falling between two adjacent levels can force every active deadline to be remapped, and only RM admits $O(1)$ insertion via one FIFO per level. Written natively, the two differ only in the ready-queue sort key. On overhead he argues the accounting is backwards: EDF pays one deadline computation per job, but RM forces far more preemptions, and his simulations show preemption counts rising roughly linearly with load under RM while *falling* under EDF at high load, because a long job holding the earliest deadline can push a successor past an arrival and avoid a preemption outright. On analysis, EDF is exact in $O(n)$ at $U \leq 1$ when deadlines equal periods, while RM’s $\ln 2 \approx 0.69$ bound needs pseudo-polynomial response-time analysis to become exact. Under permanent overload EDF rescales every period to $T_i U$ while RM can block low-priority tasks outright; under transient overload a 1.5-unit overrun in the *highest*-rate task makes τ_2 miss rather than the longest-period τ_4 , refuting the belief that RM fails predictably from the bottom up. A three-task example gives jitters of 0, 2, 8 under RM against 1, 2, 3 under EDF.

Critical judgment. The method is asymmetric, and the paper is more careful about it than its title. A counterexample can refute “RM always wins” but cannot establish “EDF wins”, and Buttazzo keeps to the weaker claim: the jitter example “does not prove that EDF always introduces less jitter... but just confutes the common belief”, and the conclusion says only that RM’s advertised properties “do not hold in general”. What overshoots is the register, not the claims: a Judgment Day that delivers a rebuttal. The overhead case is the weakest link: Section 3 counts preemptions, never measures a context switch on real hardware, and nets the saved switches against EDF’s per-job deadline update by assertion alone (“EDF introduces less runtime overhead than RM, when context switches are taken into account”), so the verdict rests on a proxy counted over synthetic task sets. Two things that would decide the industrial argument are absent: multiprocessors, and the certification ecosystem that keeps fixed priorities in use.

He does not, however, stack the deck. He grants RM the $O(1)$ ready queue and the cheaper exact test when deadlines are shorter than periods, and grants that RM confines an overrun to tasks of lower priority while EDF confines it nowhere — adding at once that the property “can be of little use if we do not know a priori which task is going to overrun”. He concludes that under *permanent* overload deciding which behaviour is better is “highly application dependent”, and that both algorithms are “not very well suited” to overload and jitter control at all, the remedy being resource reservation. That concession aged best: Linux’s `SCHED_DEADLINE` (merged in 3.14, 2014) ships EDF together with the Constant Bandwidth Server, the Abeni–Buttazzo mechanism of his own Section 7.3, rather than EDF alone.

Two of the three are acts of replacement: 2-4 unseats its own predecessor’s randomness and 2-1 unseats static rate priorities, both on the ground that a scheduler should carry the extra state needed to decide accurately — exactly the state 2-3 had traded away for a stateless draw.

References

- [1] C. A. Waldspurger and W. E. Weihl. “Lottery Scheduling: Flexible Proportional-Share Resource Management.” *Proceedings of the First USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Monterey, CA, November 1994. (Paper list ID 2-3.)
- [2] C. A. Waldspurger and W. E. Weihl. “Stride Scheduling: Deterministic Proportional-Share Resource Management.” Technical Memorandum MIT/LCS/TM-528, MIT Laboratory for Computer Science, June 1995. (Paper list ID 2-4.)
- [3] G. C. Buttazzo. “Rate Monotonic vs. EDF: Judgment Day.” *Real-Time Systems*, 29(1):5–26, 2005. (Paper list ID 2-1.)

Ex. 2.8 (22.5 pts) — Three Papers on Scheduling, Found by Search

How I found them. I searched for highly cited scheduling papers outside the course paper list whose full text is freely downloadable, then picked one from each of three eras — 1973 uniprocessor real-time theory, 2016 multicore kernel, 2019 datacentre microsecond-scale — so the three critiques would bite on different evidence rather than repeat one argument.

Paper 1 — Liu and Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment” (JACM, 1973)

Summary. The paper asks how much of one processor can be committed to m periodic tasks while still guaranteeing every deadline. A task is reduced to two numbers, its period T_i and its run-time C_i , under five assumptions: periodic requests, a deadline equal to the next request, independent tasks, constant run-times, and aperiodic work confined to initialisation and failure recovery.

For fixed priorities, Theorem 1 identifies the *critical instant* — “A critical instant for any task occurs whenever the task is requested simultaneously with requests for all higher priority tasks” — which turns an infinite schedule into a finite check. Theorem 2 shows the *rate-monotonic* rule (shorter period, higher priority, regardless of run-time) is optimal among all fixed-priority rules, and Theorem 5 bounds utilisation at $U = m(2^{1/m} - 1)$: about 0.83 for two tasks, 0.78 for three, falling to $\ln 2 \approx 0.69$. Section 7 instead assigns priorities by nearest deadline, and Theorem 7 proves that *deadline driven* rule feasible if and only if $\sum_i C_i/T_i \leq 1$, so the same hardware is schedulable at 100% load. Section 8 mixes the two, running the k shortest-period tasks rate-monotonically on the interrupt hardware; the worked example in Section 9, $T = (3, 4, 5)$ with $C_1 = C_2 = 1$, reaches 98.3% mixed against 100% deadline-driven and 78.3% rate-monotonic.

Critical judgment. The cost model is the weak point. Assumption (A4) folds context-switch cost into the constant C_i , claiming “the bookkeeping time necessary to request a successor and the costs of preemptions can be taken into account”. It cannot: how often a task is preempted depends on the whole task set, not on the task, and rate-monotonic scheduling preempts far more than deadline-driven scheduling does. The practical case for the fixed-priority algorithm is exactly the quantity the model prices at zero. Section 6 concedes that “the practical costs of switching between tasks must still be counted”, and then never counts them.

The more consequential gap is independence. The conclusion is candid about the other assumptions, calling periodicity and constant run-times “perhaps the most important and least defensible” and admitting that without periodicity “None of our analytic work would remain valid”; it even reopens the deadline assumption. Independence is examined once, and only in its easy case: (A3) “does not exclude the situation in which the occurrence of a task τ_2 can only follow a certain (fixed) number... of occurrences of a task τ_1 ”, which is absorbed by choosing commensurate periods. Fixed request-count coupling is not what real systems break on. They share mutexes, and a low-priority task holding a lock blocks a high-priority one for a time the utilisation bound has no term for. That hole stayed open until Sha, Rajkumar and Lehoczky’s priority-inheritance protocols added an explicit blocking term to the test; the 1997 Mars Pathfinder resets showed what it costs in the field. The framework itself won completely — but the paper’s parting advice, to make periodicity and constant run-times “a design goal”, is the part practice discarded, in favour of sporadic-task and server models.

Paper 2 — Lozi, Lepers, Funston, Gaud, Quéma and Fedorova, “The Linux Scheduler: a Decade of Wasted Cores” (EuroSys, 2016)

Summary. The paper takes one invariant — “make sure that ready threads are scheduled on available cores” — and shows Linux’s Completely Fair Scheduler violating it for seconds at a time. The cause is structural: per-core runqueues force periodic load balancing, balancing is too expensive to run often, and the load metric (weight scaled by utilisation, then divided by the cgroup’s thread count) is complicated enough to hide imbalance.

Four bugs follow. *Group Imbalance*: a stealing core compares the *average* load of a scheduling group, so one heavy thread masks idle cores beside it; comparing minimum load instead made a kernel `make` 13% faster and NAS `1u` 13× faster. *Scheduling Group Construction*: groups are built from core 0’s viewpoint, so nodes two hops apart land in both of the groups $\{0, 1, 2, 4, 6\}$ and $\{1, 2, 3, 4, 5, 7\}$, their averages always match and no work moves — `1u` takes 1040s instead of 38s. *Overload-on-Wakeup*: for cache reuse a waking thread is placed only among cores of the waker’s node, so it can wake onto a busy core while others idle; the fix cut TPC-H query 18 by 22.2% and the full benchmark by 13.2%. *Missing Scheduling Domains*: a call dropped during refactoring means disabling and re-enabling any core silently stops all cross-NUMA balancing, and `1u` runs 137.59× slower. None of these crash anything, so the authors add an online sanity checker for the invariant and a trace visualiser, the latter needing under 150 lines of kernel instrumentation.

Critical judgment. The bug analyses are exact, but the evidence is wide in workloads and narrow in hardware: every measurement comes from one 8-node AMD Bulldozer server. Table 4 nevertheless records “Impacted applications: All” for three of the four bugs, two of them topology-dependent — Scheduling Group Construction needs nodes two hops apart — so a claim about which *applications* suffer is doing duty as one about which *machines* do, on a sample of one.

The energy claim is weaker. The introduction asserts that “Energy waste is proportional” to the lost performance, yet power runs through the paper as a design pressure without ever being measured — no power figure, axis or table — and it bites on the Overload-on-Wakeup fix. The authors raise that tradeoff themselves rather than hiding it, since waking a thread on a long-idle core forces it out of a low-power state, but their answer is to “only enforce the new wakeup strategy if the system’s power management policy does not allow cores to enter low-power states at all”. The fix that wins most on their database workload is therefore disabled on virtually every production server, and the

energy argument that might justify re-enabling it is asserted rather than measured.

History has been kind to both forecasts. “New scheduler designs come and go” held: CFS was itself replaced, by EEVDF, in kernel 6.6 (2023). The modular alternative of Section 5 — “a scheduler that is a collection of modules: the core module and optimization modules” — was then taken up in spirit by `sched_ext` in 6.12 (2024), which moves policy into BPF programs outside the core. The fit is loose, since `sched_ext` replaces the whole scheduler rather than advising an invariant-keeping core, but the instinct is the paper’s.

Paper 3 — Kaffes, Chong, Humphries, Belay, Mazières and Kozyrakis, “Shinjuku: Preemptive Scheduling for μ second-scale Tail Latency” (NSDI, 2019)

Summary. Dataplane operating systems reach microsecond latencies by never preempting: IX hashes requests to per-core queues (distributed FCFS), and ZygOS work-steals to approximate a central FCFS queue. Both let a short request queue behind a long one, which is fatal when service times are heavy-tailed or bimodal, where theory prefers processor sharing — and processor sharing needs preemption. Linux cannot supply it, since CFS targets a 6 ms preemption latency with a 0.75 ms minimum.

Shinjuku is a single-address-space OS on Dune in which one dispatcher thread owns the queues and interrupts workers directly. Removing both VM exits from the interrupt path — posted interrupts on the receiver, a directly mapped APIC on the sender — takes the sender from 2081 to 298 cycles and the receiver from 2662 to 1212, and context switches drop to 36–109 cycles; it is that 1212-cycle receiver cost that makes preemption every $5\mu\text{s}$ affordable, at under 10 % of worker throughput. Two policies exploit it: a single queue preempting every 5–15 μs , and a multi-queue policy serving whichever queue’s head request has the largest ratio of queueing time to its SLO target. Against both IX and ZygOS, Shinjuku obtains up to 50 % lower tail latency *at low load* and $5\times$ the throughput for a given 300 μs tail-latency target on a bimodal synthetic workload, and on RocksDB serving 99.5 % GET plus 0.5 % SCAN, 88 % lower tail latency with $6.6\times$ the throughput.

Critical judgment. Centralisation is what makes the policies possible and is the claim least tested. Section 3.6 admits that one dispatcher “comfortably saturate[s] a full socket” and no more — §4.3 measures it scaling near-linearly to 11 workers at 5 M requests/s, and 9.5 M with two dispatchers — and proposes to scale further by “steer[ing] requests to different dispatchers using the NIC RSS feature”. That is the distributed queueing Section 2 blames for head-of-line blocking, reintroduced at the granularity of dispatcher groups, and what it costs is never measured: the only multi-dispatcher experiment issues “short requests with 1 μsec fixed service time to stress the dispatcher”, the one distribution under which head-of-line blocking cannot occur. The headline claim is about heavy tails; the scaling result is measured where tails are absent.

The security discussion is honest, but its remedy is still a promise. Mapping APIC registers into the process means, as the authors say, that “one process could launch a denial-of-service attack on another process by issuing a large number of interrupts to a specific core”; they answer that “future versions of Shinjuku will run application code in ring 3 ... eliminating this attack vector with very small overhead”, citing an 84-cycle ring transition. But every evaluation number was taken with applications in ring 0, chosen “to avoid the address space crossings” — crossings that, at a $5\mu\text{s}$ quantum, would recur on every preemption. The system that was measured is not the system that would be deployable.

The premise — that the obstacle was mechanism rather than policy — has held up: the hardware

the paper asks for, “support for lightweight user-level interrupts”, later shipped as Intel’s user interrupts (UINTR) in Sapphire Rapids.

References

- [1] C. L. Liu and J. W. Layland. “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment.” *Journal of the ACM*, vol. 20, no. 1, January 1973, pp. 46–61. DOI: 10.1145/321738.321743.
- [2] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma and A. Fedorova. “The Linux Scheduler: a Decade of Wasted Cores.” *Proc. 11th European Conference on Computer Systems (EuroSys ’16)*, London, April 2016. DOI: 10.1145/2901318.2901326.
- [3] K. Kaffes, T. Chong, J. T. Humphries, A. Belay, D. Mazières and C. Kozyrakis. “Shinjuku: Preemptive Scheduling for μ second-scale Tail Latency.” *Proc. 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’19)*, Boston, February 2019, pp. 345–360. <https://www.usenix.org/conference/nsdi19/presentation/kaffes>
- [4] L. Sha, R. Rajkumar and J. P. Lehoczky. “Priority Inheritance Protocols: An Approach to Real-Time Synchronization.” *IEEE Transactions on Computers*, vol. 39, no. 9, September 1990, pp. 1175–1185. (Cited above for the blocking term missing from [1].)